



Expert Racing Parts

Estrutura Completa do Projeto

- Bot ML Automático
- Site Vitrine
- Integração Bling
- Opção B — Next.js

Cliente	Expert Racing Parts
Conta Bling	integracao@expertracingparts.com.br
Conta ML	Expert.Store
Produtos	881 produtos (peças de corrida)
Data	18 de junho de 2026

1. Diagnóstico da Conta

- ✔

O QUE JÁ EXISTE (levantado via acesso ao Bling)
- ✔

881 produtos cadastrados no Bling com SKU, preço e estoque
- ✔

Tray Commerce já integrado e sincronizado com o Bling
- ✔

Amazon já integrado ao Bling
- ✔

IDERIS e Olist (hubs de marketplaces) já integrados
- ✔

EXPERT.STORE canal próprio já conectado
- ✔

Nicho definido: peças de alta performance (blocos de motor, filtros, suportes)

- ⚠

O QUE PRECISA SER CRIADO / RESOLVIDO
- ⚠

Sem acesso ao código Tray → não dá para customizar redirect para ML
- ⚠

Tray cobra mensalidade para um checkout que o cliente não vai usar
- ⚠

Bot ML ainda não existe
- ⚠

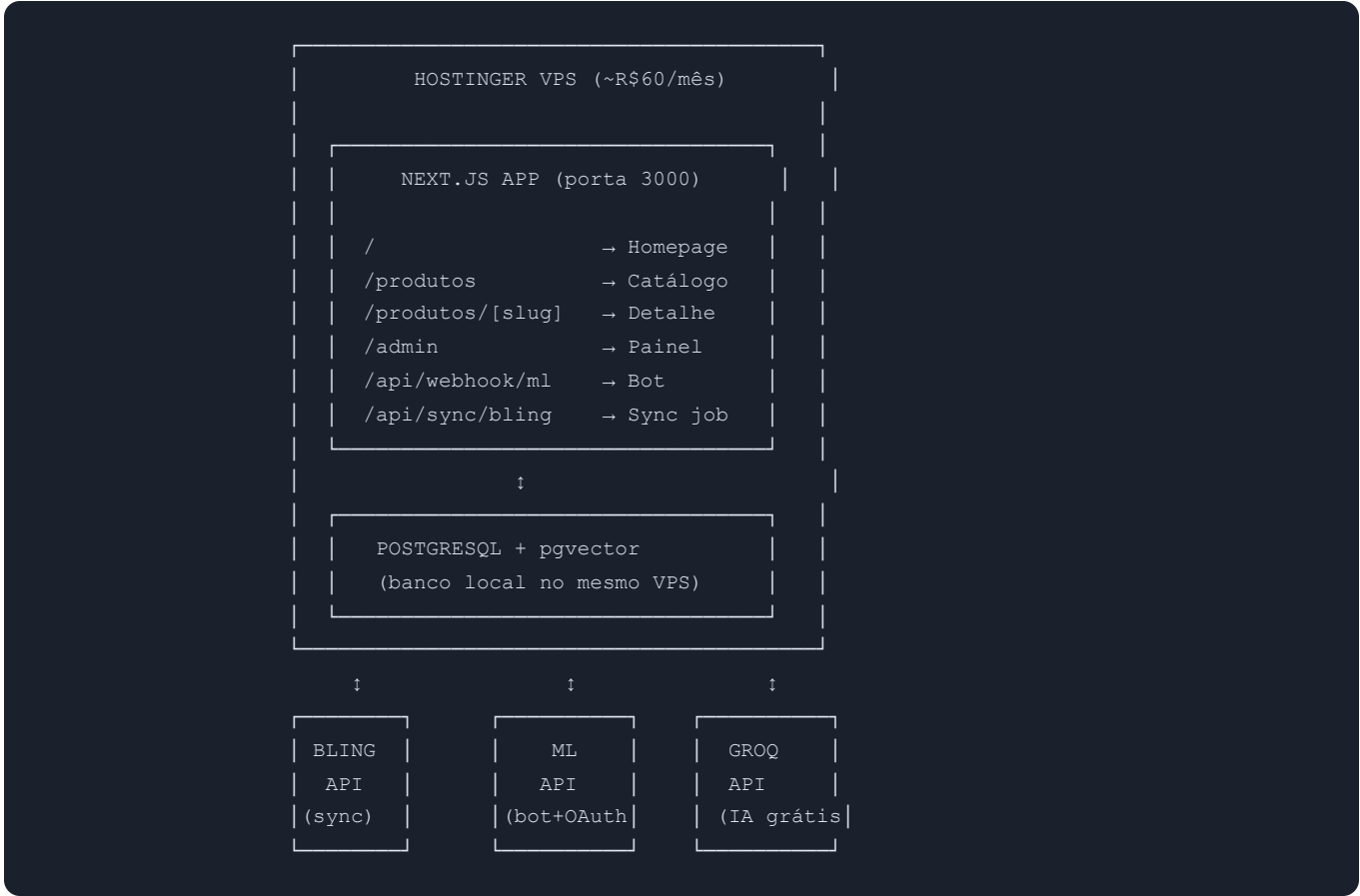
Site vitrine com redirect para ML ainda não existe
- ⚠

VPS Hostinger ainda não criado

POR QUE OPÇÃO B (NEXT.JS) E NÃO OPÇÃO A (TRAY)

Critério	Tray (Opção A)	Next.js (Opção B)
Acesso ao código	Sem acesso (SaaS)	100% controle
Redirect para ML	Difícil / impossível sem código	Nativo, 5 min
Custo mensal	R\$200–500/mês	R\$60/mês
Integração Bling	Já existe (mas limitada)	Nativa via API
Bot integrado	Precisaria de sistema separado	No mesmo servidor
Checkout	Forçado para Tray	Redireciona direto para ML

2. Arquitetura Geral do Sistema



COMPONENTES E RESPONSABILIDADES

Componente	Função	Tecnologia
Site Vitrine	Exibir catálogo, redirecionar para ML	Next.js 15
Bot ML	Receber webhook, gerar resposta com IA, postar	Node.js + Fastify
Sync Bling	Sincronizar produtos e estoque do Bling	Cron job (a cada 1h)
Painel Admin	Config bot, histórico, conectar ML	Next.js App Router
Banco de Dados	Catálogo, histórico Q&A, tokens ML	PostgreSQL + pgvector
IA	Gerar respostas contextualizadas	Groq (Llama 3.3 70B) GRÁTIS
Embeddings (RAG)	Busca semântica em histórico Q&A	Gemini text-embedding GRÁTIS

3. Estrutura de Pastas do Projeto

```
ml-bot/ └─ src/ └─ app/ # Next.js App Router └─ page.tsx # Homepage └─ produtos/ └─
└─ page.tsx # Catálogo de produtos └─ [slug]/ └─ page.tsx # Detalhes + botão "Comprar
no ML" └─ admin/ └─ page.tsx # Dashboard admin └─ historico/page.tsx # Histórico
de Q&As └─ configuracoes/page.tsx # Prompt do bot + config └─ conectar/page.tsx # OAuth
ML └─ api/ └─ webhook/ └─ questions/route.ts # Recebe webhook do ML └─ sync/ └─
└─ bling/route.ts # Trigger de sincronização └─ auth/ └─ callback/route.ts # Callback
OAuth ML └─ services/ # Lógica de negócio └─ ml-api.ts # Client ML API (perguntas,
respostas) └─ bling-api.ts # Client Bling API (produtos, estoque) └─ groq.ts # Client Groq
(gera resposta IA) └─ gemini.ts # Client Gemini (embeddings RAG) └─ rag.ts # Busca
semântica no histórico └─ bot.ts # Orquestrador: recebe → processa → responde └─ sync.ts #
Sincroniza Bling → banco └─ db/ # Banco de dados └─ schema.sql # DDL das tabelas └─
client.ts # Pool de conexão PostgreSQL └─ queries.ts # Queries reutilizáveis └─ lib/ #
Utilitários └─ auth.ts # OAuth helpers └─ crypto.ts # Encrypt/decrypt tokens └─ logger.ts #
Logs estruturados └─ .env.example # Variáveis de ambiente (template) └─ .env # NUNCA versionar no
Git └─ docker-compose.yml # PostgreSQL local para dev └─ package.json └─ README.md
```

4. Banco de Dados (PostgreSQL)

5 TABELAS PRINCIPAIS

```
-- Produtos sincronizados do Bling
CREATE TABLE products (
  id          TEXT PRIMARY KEY,          -- ml_item_id do ML
  sku         TEXT,                     -- SKU do Bling (ex: EXP3001300001-USA)
  bling_id    TEXT,                     -- ID interno do Bling
  title       TEXT NOT NULL,
  description  TEXT,
  price       DECIMAL(10,2),
  cost        DECIMAL(10,2),
  category    TEXT,
  image_url   TEXT,
  ml_item_url TEXT,                     -- URL do anúncio no ML
  custom_instructions TEXT,             -- Instruções específicas do bot para este produto
  sync_status TEXT DEFAULT 'pending', -- pending | synced | error
  last_synced_at TIMESTAMP,
  created_at  TIMESTAMP DEFAULT NOW()
);

-- Estoque (separado para atualizar sem recriar produto)
CREATE TABLE inventory (
  product_id TEXT PRIMARY KEY REFERENCES products(id),
  quantity    INT DEFAULT 0,
  reserved    INT DEFAULT 0,           -- vendidos aguardando entrega
  last_updated TIMESTAMP DEFAULT NOW()
);

-- Histórico de perguntas e respostas (base do RAG)
CREATE TABLE qa_history (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  product_id  TEXT REFERENCES products(id),
  ml_question_id TEXT UNIQUE,
  question    TEXT NOT NULL,
  answer      TEXT NOT NULL,
  embedding   vector(768),             -- embedding da pergunta para RAG
  status      TEXT DEFAULT 'success', -- success | failed | edited
  answered_at TIMESTAMP DEFAULT NOW()
);
CREATE INDEX ON qa_history USING ivfflat (embedding vector_cosine_ops);

-- Credenciais ML (token OAuth, renovado automaticamente)
CREATE TABLE ml_credentials (
  id          SERIAL PRIMARY KEY,
  seller_id   TEXT NOT NULL UNIQUE,
  access_token TEXT NOT NULL,          -- armazenado criptografado
  refresh_token TEXT NOT NULL,
  expires_at  TIMESTAMP NOT NULL,
  updated_at  TIMESTAMP DEFAULT NOW()
);

-- Log de webhooks recebidos (debug e auditoria)
CREATE TABLE webhook_logs (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
question_id TEXT,  
seller_id TEXT,  
status TEXT, -- received | processing | success | failed  
error TEXT,  
attempts INT DEFAULT 1,  
created_at TIMESTAMP DEFAULT NOW()  
);
```

5. Fluxo Completo — Bot de Respostas

- 1 **Comprador pergunta** no anúncio da Expert Racing Parts no ML
- 2 **ML dispara webhook** para o servidor: `POST /api/webhook/questions`
Payload: `{ "question_id": "q12345", "item_id": "MLB999", "seller_id": "..." }`
- 3 **Bot valida o webhook** (HMAC-SHA256) — garante que é realmente do ML, não spam
- 4 **Bot chama ML API** para buscar o texto completo da pergunta e os dados do produto anunciado
- 5 **Bot busca no banco local** a descrição do produto, estoque e instruções customizadas
`SELECT * FROM products WHERE id = 'MLB999'`
- 6 **RAG Engine** converte a pergunta em embedding (Gemini) e busca no histórico as 3–5 respostas mais similares para o mesmo produto via pgvector
- 7 **Prompt montado** para o Groq:
 - System prompt (tom de voz da Expert Racing)
 - Descrição do produto (Bling)
 - Estoque atual
 - Top 3 Q&As similares (RAG)
 - Pergunta do comprador
- 8 **Groq (Llama 3.3 70B)** gera resposta em <1 segundo — grátis, 14.400 req/dia
- 9 **Bot posta a resposta** no ML via API: `POST /answers` — comprador recebe em <2 segundos
- 10 **Q&A salvo** no banco com embedding → entra na base do RAG → próximas perguntas similares serão respondidas com esse contexto

6. Fluxo do Site Vitrine

- 1 **Visitante acessa** expertracingparts.com.br e navega pelo catálogo
- 2 **Site exibe** produtos com imagem, título, preço, estoque e especificações — todos vindo do banco de dados (sincronizado com Bling)
- 3 **Visitante clica** "Comprar no Mercado Livre" → redirecionado para mercadolivre.com.br/item/MLB999
- 4 **Checkout acontece no ML** — pagamento, frete, garantia, tudo pelo Mercado Livre

7. Fluxo de Sincronização Bling → Banco

- 1 **Setup inicial** — Bot importa os 881 produtos do Bling via API: título, SKU, descrição, preço, custo, categoria
 - 2 **Job de hora em hora** — verifica produtos modificados no Bling e atualiza o banco local (preço, descrição, estoque)
 - 3 **Webhook Bling (opcional)** — se Bling enviar webhook quando produto muda, atualização é instantânea
 - 4 **Site sempre atualizado** — exibe preço e estoque corretos em tempo real
-

8. Painel de Administração

Página	URL	Funcionalidade
Dashboard	/admin	Últimas 10 Q&As, taxa de sucesso, status do bot, quota Groq/Gemini
Conectar ML	/admin/conectar	Botão OAuth — feito uma única vez para vincular a conta Expert.Store
Histórico Q&A	/admin/historico	Tabela filtrável: Data, Produto, Pergunta, Resposta gerada, Status. Botão "Editar"
Configurações	/admin/configuracoes	Prompt global do bot, instruções por produto, tom de voz
Sincronização	/admin/sync	Status do último sync Bling, forçar sync manual, log de erros

EXEMPLO DE TELA DE CONFIGURAÇÃO DO BOT

Prompt do sistema (global):

Você é um atendente especialista em peças de alta performance da Expert Racing Parts. Responda de forma técnica mas acessível. Mencione sempre o frete grátis acima de R\$300. Não ofereça desconto sem autorização. Máximo 280 caracteres na resposta.

Instruções por produto:

Produto	Instrução específica
Bloco de Motor K24 Stage 1	Sempre mencionar compatibilidade com Honda Civic e Accord
Suporte Filtro AP	Informar que vem com parafusos de fixação inclusos no kit

9. Variáveis de Ambiente (.env)

```
# ===== MERCADO LIVRE =====
ML_APP_ID=SEU_APP_ID
ML_APP_SECRET=SEU_APP_SECRET
ML_REDIRECT_URI=https://expertracingparts.com.br/api/auth/callback

# ===== BLING =====
BLING_CLIENT_ID=SEU_CLIENT_ID
BLING_CLIENT_SECRET=SEU_CLIENT_SECRET

# ===== IA (GRATUITAS) =====
GROQ_API_KEY=gsk...           # groq.com → criar conta grátis
GEMINI_API_KEY=AIza...        # aistudio.google.com → criar chave grátis

# ===== BANCO DE DADOS =====
DATABASE_URL=postgresql://mlbot:senha@localhost:5432/mlbot

# ===== SEGURANÇA =====
JWT_SECRET=chave-aleatoria-32-chars
ENCRYPT_KEY=chave-para-criptografar-tokens
WEBHOOK_HMAC_SECRET=segredo-para-validar-webhooks-do-ml

# ===== APP =====
NODE_ENV=production
NEXT_PUBLIC_SITE_URL=https://expertracingparts.com.br
```

10. Setup do Servidor (Hostinger VPS)

ESPECIFICAÇÕES DO VPS RECOMENDADO

Recurso	Mínimo	Recomendado
vCPU	1	2
RAM	2 GB	4 GB
Disco	20 GB SSD	60 GB SSD
OS	Ubuntu 22.04 LTS	
Custo	~R\$60/mês	

PASSO A PASSO DE INSTALAÇÃO

```
# 1. Atualizar sistema
sudo apt update && sudo apt upgrade -y

# 2. Instalar Node.js 20 LTS
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs

# 3. Instalar PostgreSQL + pgvector
sudo apt install -y postgresql postgresql-contrib
sudo -u postgres psql -c "CREATE EXTENSION vector;"

# 4. Clonar o projeto
git clone https://github.com/seuuser/ml-bot.git /opt/ml-bot
cd /opt/ml-bot && npm install

# 5. Configurar banco
sudo -u postgres createdb mlbot
npm run db:migrate

# 6. Configurar variáveis de ambiente
cp .env.example .env
nano .env # preencher com as credenciais

# 7. Build e iniciar com PM2
npm run build
npm install -g pm2
pm2 start npm --name "ml-bot" -- start
pm2 save && pm2 startup

# 8. SSL com Let's Encrypt
sudo snap install certbot --classic
sudo certbot certonly --standalone -d expertracingparts.com.br
# Renovação automática via cron
```

11. Roadmap de Implementação

⚡ FASE 1 — MVP (2 semanas)

Semanas 1–2

- Setup Hostinger VPS + PostgreSQL + domínio
- Integração OAuth com Mercado Livre (painel admin)
- Webhook receiver + bot básico (Groq, sem RAG)
- Sincronização inicial Bling → banco (881 produtos)
- Site vitrine mínimo (listagem + detalhe + botão ML)
- Painel admin mínimo (dashboard + histórico)
- ☒ **Entrega: bot respondendo perguntas ao vivo no ML**

🧠 FASE 2 — RAG (1 semana)

Semana 3

- Integração Gemini embeddings
- pgvector configurado + índice de busca semântica
- RAG Engine: busca Q&As similares antes de gerar resposta
- Respostas ficam progressivamente mais precisas
- ☒ **Entrega: bot com memória e contexto histórico**

🔧 FASE 3 — Polish (1 semana)

Semana 4

- Painel admin completo (config, sync, stats)
- Site vitrine com design polido (filtros, busca, categorias)
- Retry queue para webhooks falhos
- Logs estruturados + alertas de falha
- Job de sincronização automática (cron a cada 1h)
- ☒ **Entrega: sistema pronto para produção**

📋 FASE 4 — Escala (futuro)

Conforme demanda

- Suporte a múltiplos sellers (SaaS)
- Monitoramento (Sentry, DataDog)
- PostgreSQL gerenciado (Supabase)
- Cobrança por cliente (R\$199–299/mês)

12. Custos Operacionais

Serviço	Custo	Observação
Hostinger VPS (2vCPU/4GB)	R\$60/mês	Site + Bot + Banco tudo junto
Domínio (.com.br)	R\$80 (one-time)	Registro.br ou Hostinger
SSL (Let's Encrypt)	Grátis	Renovação automática
Groq API (IA)	Grátis	14.400 req/dia → suficiente para qualquer seller
Gemini API (embeddings)	Grátis	1.500 req/min
Mercado Livre API	Grátis	OAuth + Webhooks
Bling API	Grátis	Cliente já paga o Bling

R\$60/mês

Custo total de infraestrutura (após R\$80 one-time de domínio)

13. O que o cliente precisa providenciar

Item	Status	Como obter
Bling (conta + API)	✅ Disponível	Já tem acesso. Criar App em bling.com.br/Api
Mercado Livre OAuth	❌ Pendente	Criar App em developers.mercadolivre.com
Hostinger VPS	❌ Pendente	Criar em hostinger.com.br (~R\$60/mês)
Groq API Key	❌ Pendente	Criar conta grátis em console.groq.com
Gemini API Key	❌ Pendente	Criar chave grátis em aistudio.google.com
Domínio	? Verificar	expertracingparts.com.br — já tem?

★ RESUMO EXECUTIVO:

Sistema composto por **3 partes integradas**: (1) **Bot ML** que responde perguntas em <2 segundos usando IA gratuita contextualizada com dados dos 881 produtos; (2) **Site Vitrine** que exibe o catálogo sincronizado do Bling e redireciona compradores para o ML; (3) **Painel Admin** para o cliente gerenciar tudo. Tudo hospedado em um único VPS Hostinger por R\$60/mês. MVP em 2 semanas, sistema completo em 4 semanas.